

```
!pip install numpy scipy scikit-learn matplotlib pandas
```

[Show hidden output](#)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.signal import stft
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score
)

print("Libraries imported successfully")
```

Libraries imported successfully

```
# Reproducibility
SEED = 42
np.random.seed(SEED)

# Signal parameters
FS = 2000          # Sampling frequency in Hz
DURATION = 1.0     # Signal duration in seconds
N_SAMPLES = int(FS * DURATION)

# Number of examples
SAMPLES_PER_CLASS = 200

# SNR levels
SNR_LEVELS = [-10, -5, 0, 5, 10]

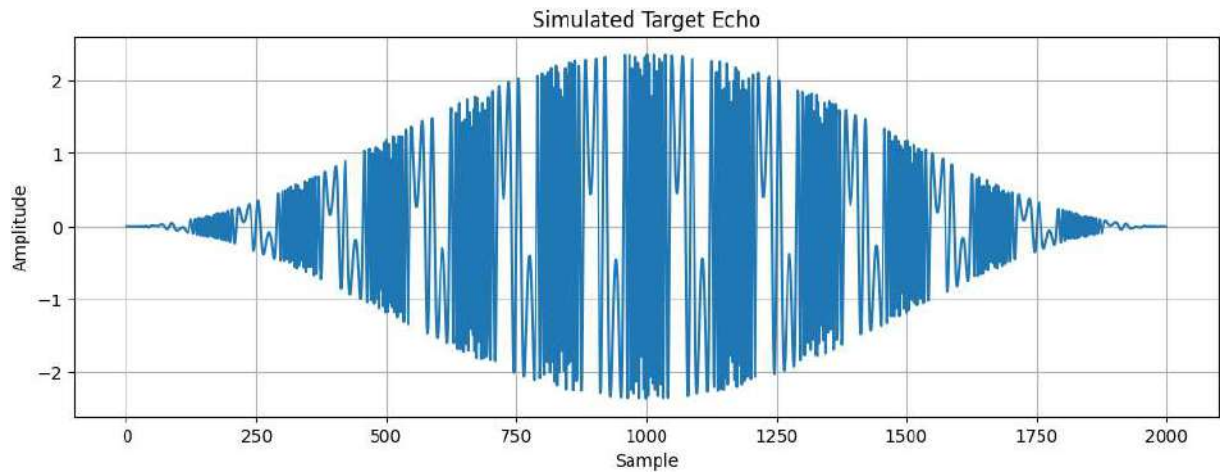
# STFT parameters
NFFT = 256
NOVERLAP = 128

print("Sampling frequency:", FS, "Hz")
print("Signal length:", N_SAMPLES)
print("SNR levels:", SNR_LEVELS)
```

```
Sampling frequency: 2000 Hz  
Signal length: 2000  
SNR levels: [-10, -5, 0, 5, 10]
```

```
def generate_target_signal(fs=FS, duration=DURATION):  
    t = np.arange(0, duration, 1/fs)  
  
    # Main target Doppler frequency  
    f0 = 180  
  
    # Micro-Doppler parameters  
    fm = 12  
    fd = 25  
  
    # Frequency modulation  
    phase = (  
        2 * np.pi * f0 * t  
        + fd * np.sin(2 * np.pi * fm * t)  
    )  
  
    target = np.sin(phase)  
  
    # Apply smooth envelope  
    envelope = np.hanning(len(t))  
  
    target = target * envelope  
  
    # Normalize  
    target = target / np.sqrt(np.mean(target**2))  
  
    return target
```

```
target = generate_target_signal()  
  
plt.figure(figsize=(12, 4))  
plt.plot(target)  
plt.title("Simulated Target Echo")  
plt.xlabel("Sample")  
plt.ylabel("Amplitude")  
plt.grid()  
plt.show()
```



```
def generate_clutter(length=N_SAMPLES):

    # Random clutter
    raw = np.random.randn(length)

    # Correlate the clutter using a simple moving average
    kernel = np.ones(25) / 25

    clutter = np.convolve(raw, kernel, mode="same")

    # Normalize
    clutter = clutter / np.sqrt(np.mean(clutter**2))

    # Random clutter envelope
    envelope = np.abs(
        np.random.normal(1.0, 0.25, length)
    )

    clutter = clutter * envelope

    # Normalize again
    clutter = clutter / np.sqrt(np.mean(clutter**2))

    return clutter
```

```
def generate_noise(length=N_SAMPLES):
    noise = np.random.randn(length)
```

```
noise = noise / np.sqrt(np.mean(noise**2))

return noise
```

```
def add_noise_for_snr(signal, snr_db):

    signal_power = np.mean(signal**2)

    noise_power = signal_power / (10 ** (snr_db / 10))

    noise = np.random.randn(len(signal))

    noise = noise / np.sqrt(np.mean(noise**2))

    noise = noise * np.sqrt(noise_power)

    return signal + noise
```

```
def generate_return(target_present=True, snr_db=0):

    clutter = generate_clutter()
    noise = generate_noise()

    if target_present:
        target = generate_target_signal()

        # Weak target relative to clutter
        target = 0.5 * target

        signal = target + clutter

    else:
        signal = clutter

    received = add_noise_for_snr(signal, snr_db)

    return received
```

```
snr = -5

target_return = generate_return(
    target_present=True,
    snr_db=snr
)

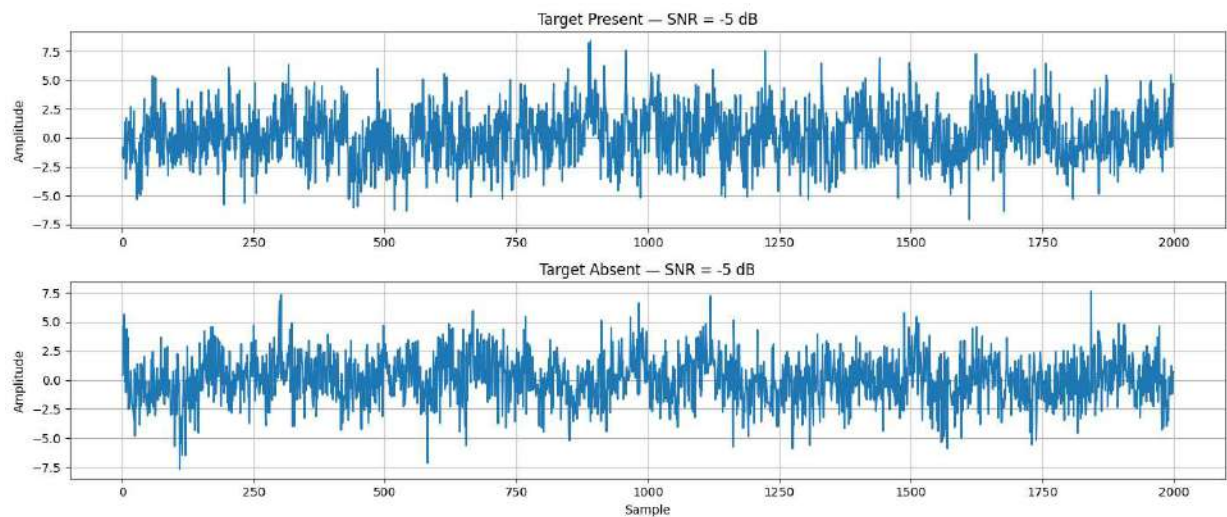
no_target_return = generate_return(
    target_present=False,
    snr_db=snr
)
```

```
plt.figure(figsize=(14, 6))

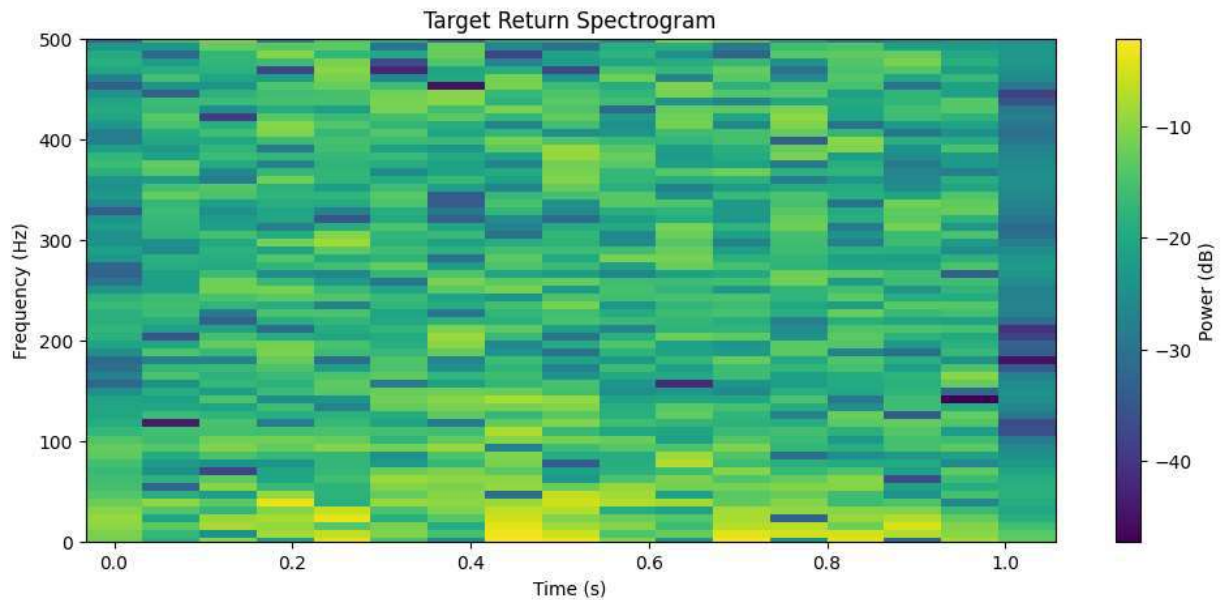
plt.subplot(2, 1, 1)
plt.plot(target_return)
plt.title(f"Target Present — SNR = {snr} dB")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2, 1, 2)
plt.plot(no_target_return)
plt.title(f"Target Absent — SNR = {snr} dB")
plt.xlabel("Sample")
plt.ylabel("Amplitude")
plt.grid()

plt.tight_layout()
plt.show()
```



```
def compute_spectrogram(signal):  
  
    frequencies, times, Zxx = stft(  
        signal,  
        fs=FS,  
        nperseg=NFFT,  
        noverlap=NOVERLAP  
    )  
  
    power = np.abs(Zxx) ** 2  
  
    return frequencies, times, power  
  
frequencies, times, power = compute_spectrogram(target_return)  
  
plt.figure(figsize=(12, 5))  
  
plt.pcolormesh(  
    times,  
    frequencies,  
    10 * np.log10(power + 1e-12),  
    shading="auto"  
)  
  
plt.colorbar(label="Power (dB)")  
plt.xlabel("Time (s)")  
plt.ylabel("Frequency (Hz)")  
plt.title("Target Return Spectrogram")  
plt.ylim(0, 500)  
  
plt.show()
```



```
def spectral_concentration(power, frequencies):

    spectrum = np.mean(power, axis=1)

    total_power = np.sum(spectrum)

    band = spectrum[
        (frequencies >= 100) &
        (frequencies <= 300)
    ]

    concentration = np.sum(band) / (total_power + 1e-12)

    return concentration
```

```
def peak_to_mean(power):

    peak = np.max(power)
    mean = np.mean(power)

    return peak / (mean + 1e-12)
```

```
def doppler_bandwidth(power, frequencies):  
  
    spectrum = np.mean(power, axis=1)  
  
    total = np.sum(spectrum)  
  
    mean_frequency = np.sum(  
        frequencies * spectrum  
    ) / (total + 1e-12)  
  
    variance = np.sum(  
        ((frequencies - mean_frequency) ** 2)  
        * spectrum  
    ) / (total + 1e-12)  
  
    bandwidth = np.sqrt(variance)  
  
    return bandwidth
```

```
def spectral_entropy(power):  
  
    spectrum = np.mean(power, axis=1)  
  
    probability = spectrum / (  
        np.sum(spectrum) + 1e-12  
    )  
  
    entropy = -np.sum(  
        probability *  
        np.log2(probability + 1e-12)  
    )  
  
    return entropy
```

```
def extract_features(signal):  
  
    frequencies, times, power = compute_spectrogram(signal)  
  
    f1 = spectral_concentration(  
        power,  
        frequencies  
    )  
  
    f2 = peak_to_mean(power)  
  
    f3 = doppler_bandwidth(  
        power,  
        frequencies  
    )
```



```
f4 = spectral_entropy(power)

return np.array([
    f1,
    f2,
    f3,
    f4
])
```

```
features = extract_features(target_return)

print("Feature vector:")
print(features)

print("Number of features:", len(features))
```

```
Feature vector:
[1.59875184e-01 2.17857632e+01 3.22108083e+02 6.70785085e+00]
Number of features: 4
```

```
def create_dataset(snr_db, samples_per_class=200):

    X = []
    y = []

    # Target-present samples
    for _ in range(samples_per_class):

        signal = generate_return(
            target_present=True,
            snr_db=snr_db
        )

        features = extract_features(signal)

        X.append(features)
        y.append(1)

    # Target-absent samples
    for _ in range(samples_per_class):

        signal = generate_return(
            target_present=False,
            snr_db=snr_db
        )

        features = extract_features(signal)

        X.append(features)
        y.append(0)
```

```
return np.array(X), np.array(y)
```

```
datasets = {}

for snr in SNR_LEVELS:

    print(f"Generating data for SNR = {snr} dB...")

    X, y = create_dataset(
        snr,
        samples_per_class=SAMPLES_PER_CLASS
    )

    datasets[snr] = (X, y)

print("Dataset generation complete.")
```

```
Generating data for SNR = -10 dB...
Generating data for SNR = -5 dB...
Generating data for SNR = 0 dB...
Generating data for SNR = 5 dB...
Generating data for SNR = 10 dB...
Dataset generation complete.
```

```
for snr in SNR_LEVELS:

    X, y = datasets[snr]

    print(
        f"SNR {snr:>3} dB : "
        f"X shape = {X.shape}, "
        f"y shape = {y.shape}"
    )
```

```
SNR -10 dB : X shape = (400, 4), y shape = (400,)
SNR  -5 dB : X shape = (400, 4), y shape = (400,)
SNR   0 dB : X shape = (400, 4), y shape = (400,)
SNR   5 dB : X shape = (400, 4), y shape = (400,)
SNR  10 dB : X shape = (400, 4), y shape = (400,)
```

```
X, y = datasets[0]

feature_names = [
    "Spectral Concentration",
    "Peak-to-Mean",
    "Doppler Bandwidth",
    "Spectral Entropy"
]
```

```
df = pd.DataFrame(
    X,
    columns=feature_names
)

df["Target"] = y

df.head(10)
```

	Spectral Concentration	Peak-to- Mean	Doppler Bandwidth	Spectral Entropy	Target
0	0.146418	40.334526	311.991940	6.142062	1
1	0.171665	37.464337	311.916377	6.290405	1
2	0.152579	42.091271	308.733819	6.142258	1
3	0.157341	63.270186	305.575737	6.196186	1
4	0.147819	50.079733	316.120019	6.105829	1
5	0.165666	36.938270	304.096205	6.172841	1
6	0.159519	68.274616	313.848902	6.142455	1
7	0.156241	42.519668	304.063050	6.162708	1
8	0.156279	100.976143	307.377532	6.098595	1
9	0.165094	68.688812	310.473050	6.071729	1



Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
def prepare_features(X_train, X_test):

    scaler = StandardScaler()

    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    pca = PCA(
        n_components=4,
        random_state=SEED
    )

    X_train_pca = pca.fit_transform(
        X_train_scaled
    )

    X_test_pca = pca.transform(
        X_test_scaled
    )
```

```
    return (  
        X_train_pca,  
        X_test_pca,  
        scaler,  
        pca  
    )
```

```
def train_rbf_svm(X_train, y_train):
```

```
    model = SVC(  
        kernel="rbf",  
        C=10,  
        gamma="scale",  
        probability=True,  
        random_state=SEED  
    )  
  
    model.fit(  
        X_train,  
        y_train  
    )  
  
    return model
```

```
results = []  
  
for snr in SNR_LEVELS:  
  
    print(f"\nProcessing SNR = {snr} dB")  
  
    X, y = datasets[snr]  
  
    # Same split every time  
    X_train, X_test, y_train, y_test = train_test_split(  
        X,  
        y,  
        test_size=0.30,  
        random_state=SEED,  
        stratify=y  
    )  
  
    # Scaling + PCA  
    (  
        X_train_pca,  
        X_test_pca,  
        scaler,  
        pca  
    ) = prepare_features(  
        X_train,  
        X_test
```

```
)

# RBF-SVM
model = train_rbf_svm(
    X_train_pca,
    y_train
)

# Prediction
y_pred = model.predict(
    X_test_pca
)

y_prob = model.predict_proba(
    X_test_pca
)[: , 1]

# Metrics
accuracy = accuracy_score(
    y_test,
    y_pred
)

auc = roc_auc_score(
    y_test,
    y_prob
)

results.append({
    "SNR_dB": snr,
    "Accuracy": accuracy,
    "AUC": auc
})

results_df = pd.DataFrame(results)

results_df
```

Processing SNR = -10 dB

Processing SNR = -5 dB

Processing SNR = 0 dB

Processing SNR = 5 dB

Processing SNR = 10 dB

	SNR_dB	Accuracy	AUC	
0	-10	0.608333	0.657222	
1	-5	0.791667	0.861111	
2	0	0.975000	0.997500	
3	5	0.975000	0.998056	
4	10	0.975000	0.998889	

Next steps:

[Generate code with results_df](#)

[New interactive sheet](#)

```
plt.figure(figsize=(9, 6))

plt.plot(
    results_df["SNR_dB"],
    results_df["Accuracy"],
    marker="o",
    linewidth=2
)

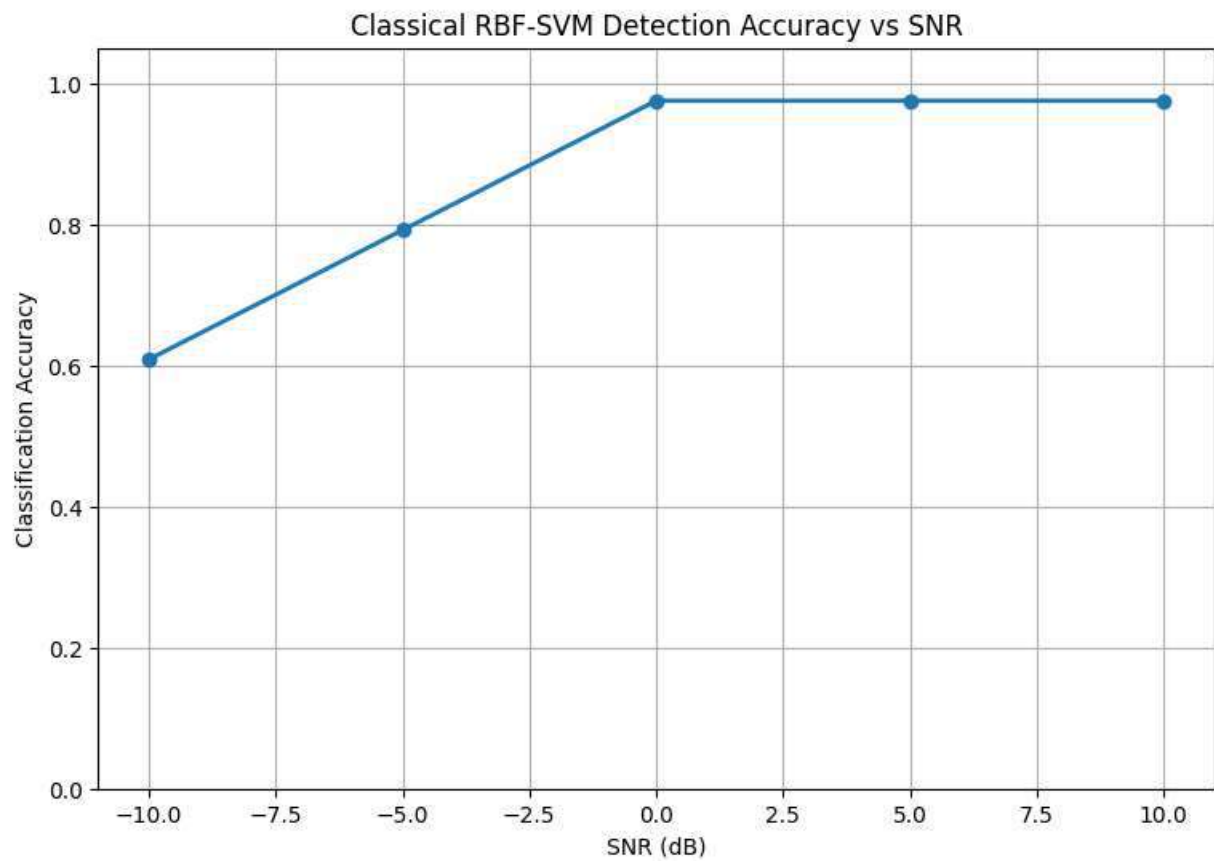
plt.xlabel("SNR (dB)")
plt.ylabel("Classification Accuracy")

plt.title(
    "Classical RBF-SVM Detection Accuracy vs SNR"
)

plt.grid(True)

plt.ylim(0, 1.05)

plt.show()
```



```
snr = 0

X, y = datasets[snr]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=SEED,
    stratify=y
)

X_train_pca, X_test_pca, scaler, pca = prepare_features(
    X_train,
    X_test
)

model = train_rbf_svm(
    X_train_pca,
    y_train
```

```
)

y_pred = model.predict(X_test_pca)

cm = confusion_matrix(
    y_test,
    y_pred
)

print(cm)
```

```
[[59  1]
 [ 2 58]]
```

```
plt.figure(figsize=(6, 5))

plt.imshow(cm)

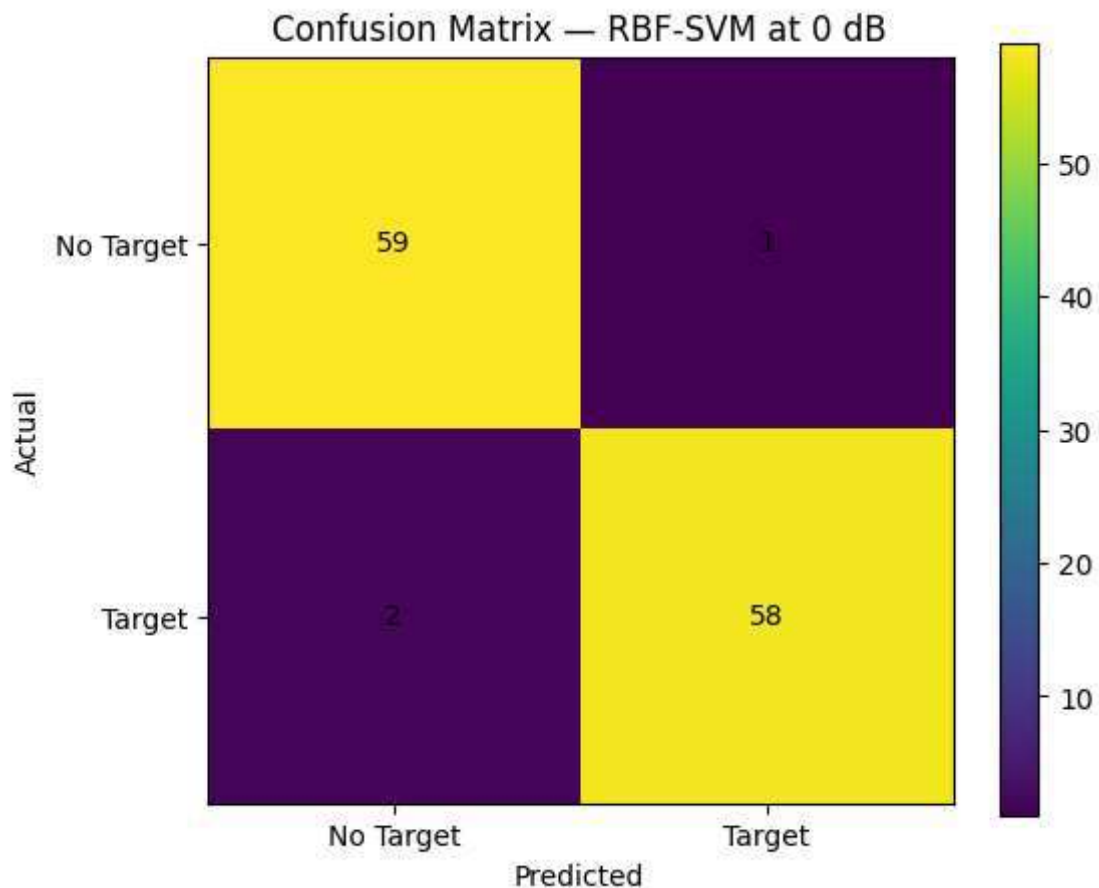
plt.title("Confusion Matrix – RBF-SVM at 0 dB")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.xticks(
    [0, 1],
    ["No Target", "Target"]
)

plt.yticks(
    [0, 1],
    ["No Target", "Target"]
)

for i in range(2):
    for j in range(2):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.colorbar()
plt.show()
```

```

y_prob = model.predict_proba(
    X_test_pca
)[: , 1]

fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob
)

auc = roc_auc_score(
    y_test,
    y_prob
)

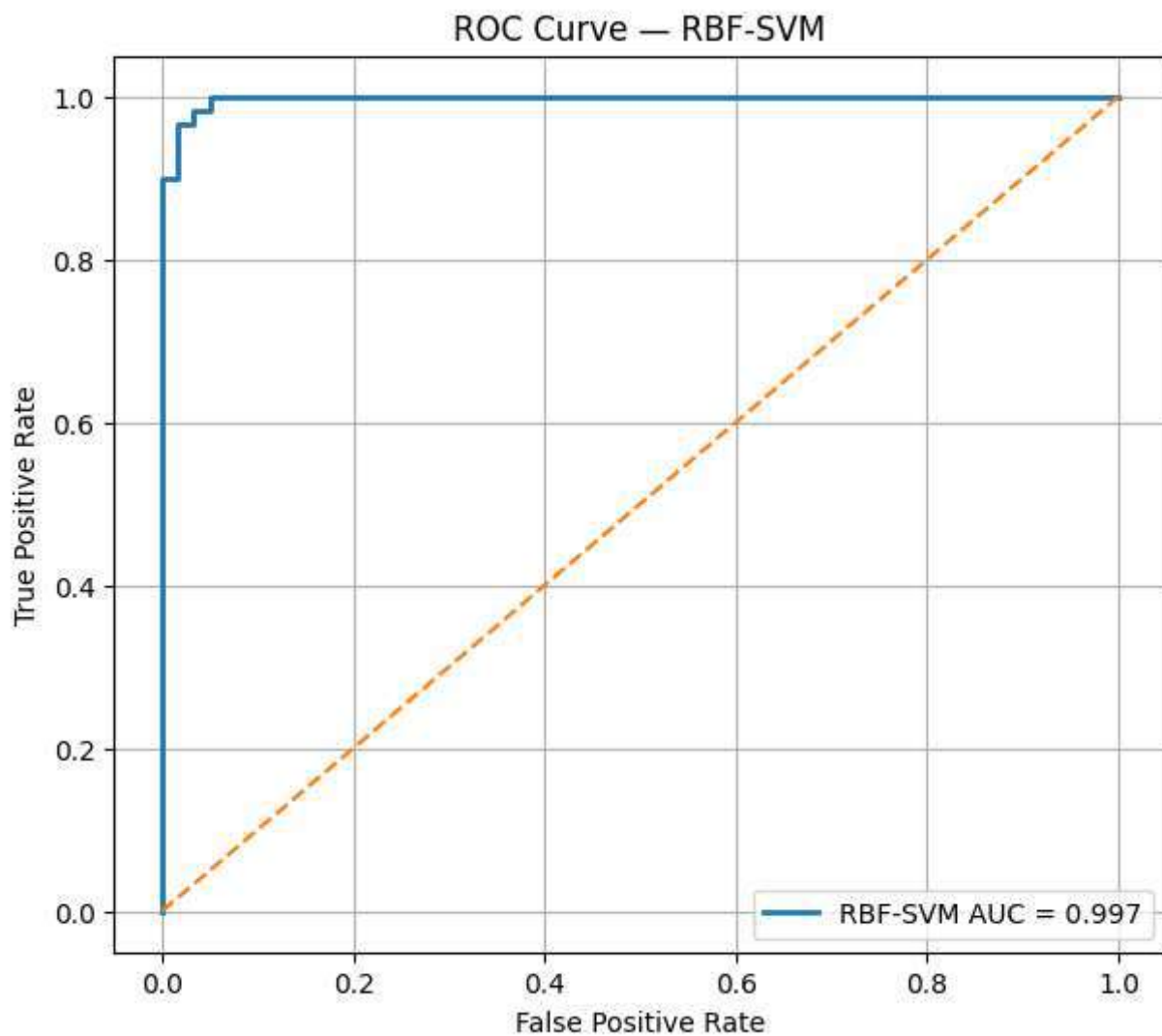
plt.figure(figsize=(7, 6))

plt.plot(
    fpr,
    tpr,
    linewidth=2,
    label=f"RBF-SVM AUC = {auc:.3f}"
)

plt.plot(
    [0, 1],

```

```
[0, 1],  
linestyle="--")  
)  
  
plt.xlabel("False Positive Rate")  
plt.ylabel("True Positive Rate")  
  
plt.title("ROC Curve — RBF-SVM")  
  
plt.legend()  
plt.grid()  
  
plt.show()
```



```
def matched_filter_score(signal, template):  
  
    signal = signal - np.mean(signal)  
    template = template - np.mean(template)  
  
    correlation = np.correlate(  
        signal,
```

```

        template,
        mode="valid"
    )

    score = np.max(
        np.abs(correlation)
    )

    normalization = (
        np.linalg.norm(signal) *
        np.linalg.norm(template)
        + 1e-12
    )

    return score / normalization

```

```

template = generate_target_signal()

print("Matched filter template created.")

```

Matched filter template created.

```

for snr in SNR_LEVELS:

    target_signal = generate_return(
        target_present=True,
        snr_db=snr
    )

    no_target_signal = generate_return(
        target_present=False,
        snr_db=snr
    )

    target_score = matched_filter_score(
        target_signal,
        template
    )

    no_target_score = matched_filter_score(
        no_target_signal,
        template
    )

    print(
        f"SNR {snr:>3} dB | "
        f"Target score = {target_score:.4f} | "
        f"No-target score = {no_target_score:.4f}"
    )

```

SNR -10 dB	Target score = 0.1545	No-target score = 0.0013
SNR -5 dB	Target score = 0.2133	No-target score = 0.0235
SNR 0 dB	Target score = 0.2888	No-target score = 0.0293
SNR 5 dB	Target score = 0.3698	No-target score = 0.0360
SNR 10 dB	Target score = 0.4525	No-target score = 0.0037

```
def matched_filter_dataset(
    signals,
    template
):

    scores = []

    for signal in signals:

        score = matched_filter_score(
            signal,
            template
        )

        scores.append(score)

    return np.array(scores)
```

```
def create_raw_dataset(
    snr_db,
    samples_per_class=200
):

    signals = []
    labels = []

    for _ in range(samples_per_class):

        signal = generate_return(
            target_present=True,
            snr_db=snr_db
        )

        signals.append(signal)
        labels.append(1)

    for _ in range(samples_per_class):

        signal = generate_return(
            target_present=False,
            snr_db=snr_db
        )

        signals.append(signal)
```

```
        labels.append(0)

    return (
        np.array(signals),
        np.array(labels)
    )
```

```
raw_datasets = {}

for snr in SNR_LEVELS:

    signals, labels = create_raw_dataset(
        snr,
        samples_per_class=200
    )

    raw_datasets[snr] = (
        signals,
        labels
    )
```

```
matched_results = []

for snr in SNR_LEVELS:

    signals, labels = raw_datasets[snr]

    scores = matched_filter_dataset(
        signals,
        template
    )

    # Split scores into train/test
    (
        scores_train,
        scores_test,
        y_train,
        y_test
    ) = train_test_split(
        scores,
        labels,
        test_size=0.30,
        random_state=SEED,
        stratify=labels
    )

    # Find threshold from training scores
    positive_scores = scores_train[y_train == 1]
    negative_scores = scores_train[y_train == 0]
```

```

threshold = (
    np.mean(positive_scores)
    + np.mean(negative_scores)
) / 2

y_pred = (
    scores_test >= threshold
).astype(int)

accuracy = accuracy_score(
    y_test,
    y_pred
)

matched_results.append({
    "SNR_dB": snr,
    "Accuracy": accuracy
})

matched_df = pd.DataFrame(
    matched_results
)

matched_df

```

	SNR_dB	Accuracy	
0	-10	1.0	
1	-5	1.0	
2	0	1.0	
3	5	1.0	
4	10	1.0	

Next steps:

[Generate code with matched_df](#)[New interactive sheet](#)

```

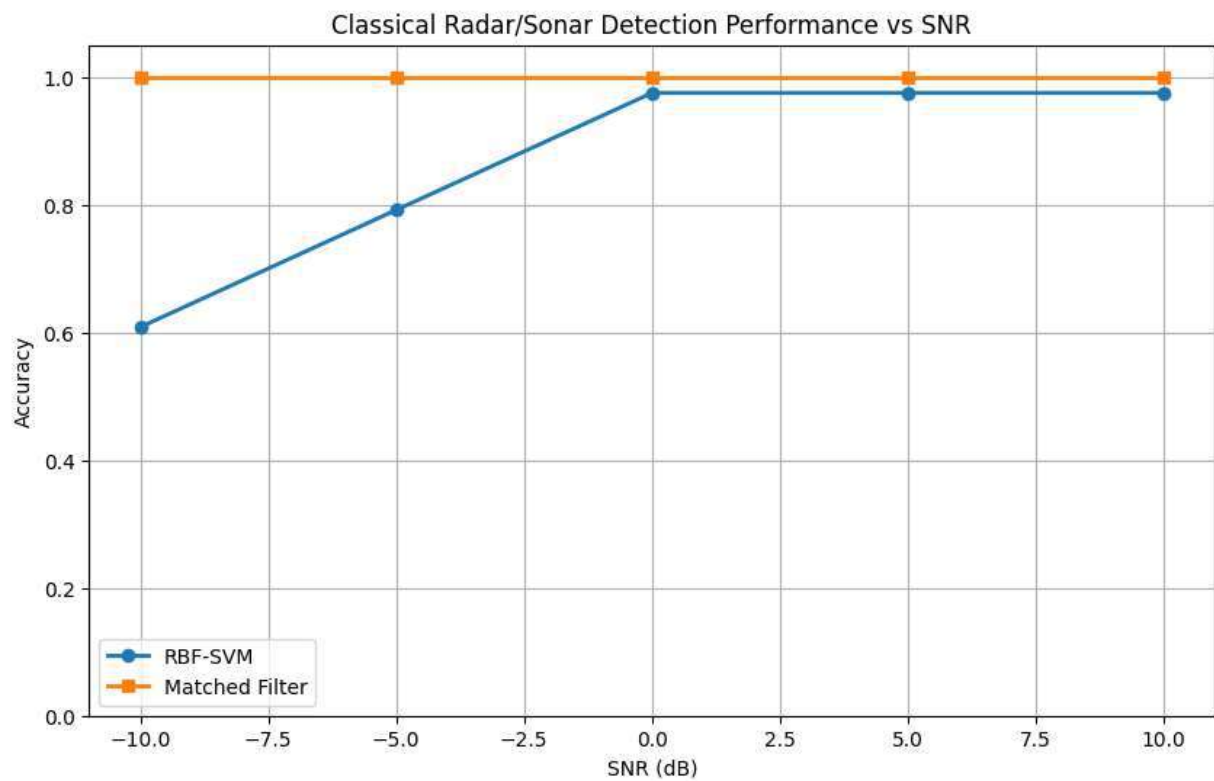
plt.figure(figsize=(10, 6))

plt.plot(
    results_df["SNR_dB"],
    results_df["Accuracy"],
    marker="o",
    linewidth=2,
    label="RBF-SVM"
)

plt.plot(
    matched_df["SNR_dB"],

```

```
matched_df["Accuracy"],  
marker="s",  
linewidth=2,  
label="Matched Filter"  
)  
  
plt.xlabel("SNR (dB)")  
plt.ylabel("Accuracy")  
  
plt.title(  
    "Classical Radar/Sonar Detection Performance vs SNR"  
)  
  
plt.grid(True)  
  
plt.legend()  
  
plt.ylim(0, 1.05)  
  
plt.show()
```



```
results_df.to_csv(
    "week1_rbf_svm_results.csv",
    index=False
)

matched_df.to_csv(
    "week1_matched_filter_results.csv",
    index=False
)

print("Results saved.")
```

Results saved.

```
plt.figure(figsize=(10, 6))

plt.plot(
    results_df["SNR_dB"],
    results_df["Accuracy"],
    marker="o",
    linewidth=2,
    label="RBF-SVM"
)

plt.plot(
    matched_df["SNR_dB"],
    matched_df["Accuracy"],
    marker="s",
    linewidth=2,
    label="Matched Filter"
)

plt.xlabel("SNR (dB)")
plt.ylabel("Accuracy")

plt.title(
```